

Edge-to-Cloud Telemetry Ingestion Pipeline

Emerson AspenTech - Senior Principal Software Developer
System Design Interview Guide

Problem: Design an ingestion architecture for thousands of industrial edge devices producing high-frequency time-series data under volatile, intermittent network conditions.

Recommended interview order

- **1. Requirements:** establish scale, reliability, latency, ordering and offline behavior.
- **2. HLD:** separate edge durability, unreliable WAN and cloud-scale ingestion.
- **3. Failure paths:** network outage, crash, duplicate, reconnect storm, cloud slowdown and disk full.
- **4. LLD:** WAL, batching, ACK/checkpoint, retry, deduplication and backpressure.
- **5. Code:** show a small C++11/14 implementation of the edge store-and-forward path.

Core design sentence: Because the WAN is intermittent, make the edge durable. Because retries create duplicates, make cloud processing idempotent. Because cloud traffic is bursty, decouple ingestion from downstream processing using a durable event stream.

1. Functional Requirements

Start here in the interview. Do not start by naming Kafka, MQTT or a database.

ID	Requirement	Meaning in this design
FR1	Continuous collection	Collect telemetry from sensors/PLCs even while the WAN is unavailable.
FR2	Store-and-forward	Persist unsent telemetry locally and replay it after connectivity returns.
FR3	Near-real-time ingestion	When connected, deliver important telemetry to cloud with low latency.
FR4	Reliable delivery	Do not silently lose critical telemetry because of transient network/process failures.
FR5	Duplicate handling	Retries must not cause duplicate logical measurements downstream.
FR6	Ordering	Preserve ordering where required, normally per device/asset rather than globally.
FR7	Historical storage	Persist time-series data for queries, analytics and predictive maintenance.
FR8	Multiple consumers	Support alerting, dashboards, analytics, ML and archival independently.
FR9	Priority	Critical alarms can bypass or outrank bulk diagnostic telemetry.

Example telemetry record

```
DeviceId: Pump-101
Sequence: 884201
EventTime: 2026-10-01T10:01:01.021
Metric: vibration
Value: 12.3
Quality: GOOD
Checksum: ...
```

The important fields for reliability are **device identity + sequence number + event timestamp**. The sequence number helps detect replay/duplicates; event time preserves when the physical measurement actually occurred.

2. Non-Functional Requirements

Area	What to establish / design for
Scale	Thousands to tens of thousands of devices; potentially hundreds of thousands to millions of measurements/sec.
Availability	Cloud ingestion must tolerate instance/broker failures; edge must continue collecting during WAN outage.
Durability	Critical records survive edge process restart/reboot and cloud consumer failures.
Latency	Low latency when online; bounded batching delay for normal telemetry; alarms may use a faster path.
Backpressure	Cloud slowdown must not cause unbounded memory growth or cascading failure.
Security	Per-device identity, TLS/mTLS, certificate rotation/revocation, authorization and encryption at rest.
Observability	Queue depth, oldest unsent event, reconnects, retry rate, consumer lag, duplicate rate, end-to-end latency.
Operability	Schema evolution, remote configuration, capacity limits, upgrades and replay/reprocessing.

Capacity calculation - do this early

Assume:

10,000 devices

100 measurements / second / device

200 bytes / measurement

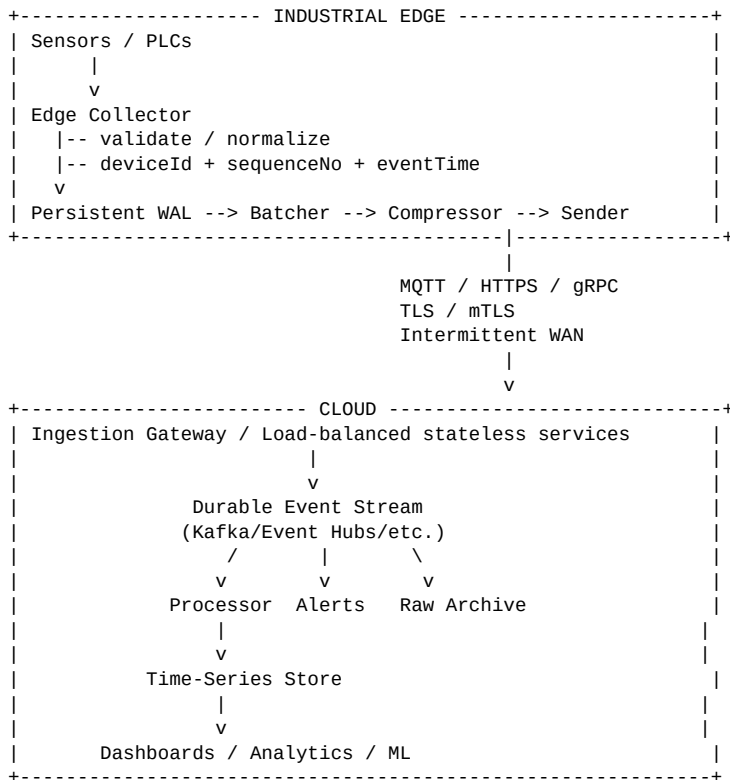
Events/sec = $10,000 * 100 = 1,000,000$ events/sec

Raw ingress ~ 200 MB/sec

Then design for bursts above average, especially when many offline gateways reconnect and replay accumulated data.

Principal-level point: ask for maximum offline duration. It directly determines required edge disk capacity.

3. High-Level Design (HLD)



There are two reliability boundaries:

- **Edge durability:** protects against WAN outage and edge process restart.
- **Cloud durable stream:** protects ingestion from slow/down downstream processors and allows independent consumers.

Why not Edge -> Database directly?

A database outage or slowdown would immediately propagate to every connected device. A durable event stream absorbs bursts, retains events while consumers recover, and decouples ingestion from storage/analytics.

4. Key Runtime Flows

4.1 Normal path

```
Sensor -> WAL -> Batch -> Send -> Cloud durable receipt -> ACK
      ^                                     |
      +----- advance checkpoint <-----+
```

After acknowledgement, old WAL segments become reclaimable.

4.2 WAN outage

```
Sensor -> WAL -> Sender -X-> Cloud
      |
      +--> records continue accumulating on local disk
```

WAN restored:

WAL backlog -> rate-limited replay -> Cloud -> ACK -> checkpoint

4.3 Lost ACK / duplicate

```
Edge                               Cloud
seq=101 -----> durable
      <----- ACK ----X (ACK lost)

seq=101 -----> retry
```

Cloud recognizes (deviceId, sequenceNo) and treats replay idempotently.

4.4 Reconnection storm

If a plant outage disconnects 5,000 gateways, all may return with a large backlog. Use exponential backoff + random jitter + server-side throttling + bounded replay rate so historical replay does not starve current critical telemetry.

5. LLD - Persistent WAL and Store-and-Forward

A WAL is an append-oriented durable log. The edge records telemetry locally **before** depending on WAN delivery.

```
wal-000001.log  seq 1      - 10,000  ACKed -> delete/reclaim
wal-000002.log  seq 10,001 - 20,000  ACKed -> delete/reclaim
wal-000003.log  seq 20,001 - 30,000  partially ACKed
wal-000004.log  seq 30,001 - ...    active append segment
```

```
checkpoint.meta: lastDurablyAckedSequence = 25,400
```

Suggested WAL record

Header:

```
magic/version
recordLength
checksum
```

Payload:

```
deviceId
sequenceNo
eventTime
metricId
value/encoded payload
quality flags
```

Durability trade-off

A successful `write()` can still leave bytes in the OS page cache. If power-loss durability matters, use `fsync()`/`fdatasync()` or an equivalent storage guarantee. Do not necessarily sync every measurement; group commit gives a throughput-vs-durability trade-off.

Recovery: on restart, scan/validate WAL segments, discard or truncate a partial/corrupt tail record, load the checkpoint, then resume sending unacknowledged records.

6. LLD - Components and Responsibilities

Component	Responsibility
Collector	Accept sensor samples; validate minimum required fields.
Normalizer	Convert units/schema; attach device identity, sequence and timestamps.
WAL Writer	Append records durably; rotate segments; recover after crash.
Batcher	Build batches by size OR maximum wait time.
Compressor	Compress larger telemetry batches where CPU/bandwidth trade-off is favorable.
Sender	Maintain connection; transmit batches; parse ACKs; retry failures.
Retry Controller	Exponential backoff, jitter, retry budget and reconnect policy.
Checkpoint Manager	Persist highest durably acknowledged sequence/range.
Retention Manager	Delete fully ACKed segments; enforce disk capacity/priority policy.
Cloud Ingestor	Authenticate, validate, rate-limit and append to durable event stream.
Stream Processor	Deduplicate, validate/enrich, aggregate and route.

Batch policy

```
Flush when:  
    record_count >= MAX_BATCH_RECORDS  
OR elapsed_time >= MAX_BATCH_DELAY  
OR critical_alarm_requires_immediate_send
```

This avoids waiting forever at low traffic while reducing per-message network/TLS/protocol overhead at high traffic.

Ordering

Partition the cloud event stream by **deviceid** (or asset/site if that is the required ordering domain). This allows horizontal scale while preserving per-device order. Avoid requiring global ordering unless the business requirement truly needs it.

7. Failure Handling and Backpressure

Failure	Design response
WAN down	Continue append to WAL; sender backs off; replay after reconnect.
Edge process crash	Restart, recover WAL/checkpoint, resend unacknowledged data.
ACK lost	Retry; cloud deduplicates using stable event identity.
Cloud ingestion unavailable	Keep locally; backoff + jitter; bounded reconnect attempts.
Broker/consumer slow	Cloud queue absorbs backlog; monitor consumer lag; scale consumers.
Time-series DB down	Continue retaining in durable event stream; consumers resume later.
Edge disk near full	Prioritize critical data; aggregate/expire explicitly permitted low-value data; alert.
Corrupt record	Checksum/validation; quarantine or skip according to policy; never silently hide corruption.
Clock drift	Keep event time and ingestion time; detect drift; use NTP/PTP where required.
Reconnect storm	Jitter, throttling, admission control, replay quotas and autoscaling.

Backpressure chain

```
Cloud overloaded
  |
  v
Ingestion throttles / rejects temporarily
  |
  v
Edge sender reduces rate
  |
  v
Persistent WAL grows (bounded by disk policy)
  |
  +--> prioritize critical alarms
  +--> optionally aggregate/drop ONLY explicitly lossy classes
```

8. C++11/14 - Simplified Edge Design

Interview code should demonstrate the control flow rather than implement a production database. This skeleton shows WAL-before-send, batching, ACK checkpointing and retry.

```
struct TelemetryRecord {
    std::string deviceId;
    uint64_t sequenceNo;
    int64_t eventTimeMs;
    std::string metric;
    double value;
};

class Wal {
public:
    // Append must complete before the record is eligible for network send.
    bool append(const TelemetryRecord& r);

    // Read records strictly after the durable ACK checkpoint.
    std::vector<TelemetryRecord> readAfter(
        uint64_t checkpoint, size_t maxRecords);

    bool persistCheckpoint(uint64_t sequenceNo);
    uint64_t loadCheckpoint() const;

    // Delete/compact segments that are fully <= checkpoint.
    void reclaim(uint64_t checkpoint);
};

class CloudClient {
public:
    struct Result {
        bool success;
        uint64_t highestDurablyStoredSequence;
    };

    Result send(const std::vector<TelemetryRecord>& batch);
};
```

9. C++11/14 - Store-and-Forward Loop

```
class EdgeAgent {
public:
    EdgeAgent(Wal& wal, CloudClient& cloud)
        : wal_(wal), cloud_(cloud),
          checkpoint_(wal.loadCheckpoint()) {}

    bool ingest(const TelemetryRecord& record) {
        // Never depend on WAN availability to accept the sample.
        return wal_.append(record);
    }

    void flushOnce() {
        const size_t kBatchSize = 500;

        std::vector<TelemetryRecord> batch =
            wal_.readAfter(checkpoint_, kBatchSize);

        if (batch.empty())
            return;

        CloudClient::Result result = cloud_.send(batch);

        if (!result.success) {
            // Do NOT remove records.
            // Retry controller will apply backoff + jitter.
            return;
        }

        // ACK means the cloud has durably accepted through this sequence.
        if (result.highestDurablyStoredSequence > checkpoint_) {
            checkpoint_ = result.highestDurablyStoredSequence;

            // Persist checkpoint before reclaiming WAL storage.
            if (wal_.persistCheckpoint(checkpoint_)) {
                wal_.reclaim(checkpoint_);
            }
        }
    }

private:
    Wal& wal_;
    CloudClient& cloud_;
    uint64_t checkpoint_;
};
```

Important: production code needs checksums, segment rotation, partial-write recovery, synchronization, batching timers, cancellation, retry policy, disk quotas, metrics and careful checkpoint/WAL atomicity.

10. C++11/14 - Retry with Exponential Backoff + Jitter

```
#include <algorithm>
#include <chrono>
#include <random>
#include <thread>

void retryDelay(unsigned attempt) {
    const unsigned kBaseMs = 200;
    const unsigned kMaxMs = 30000;

    // Cap shift to avoid overflow.
    unsigned shift = std::min(attempt, 10u);
    unsigned delay = std::min(kBaseMs * (1u << shift), kMaxMs);

    static std::mt19937 rng(
        static_cast<unsigned>(
            std::chrono::steady_clock::now()
                .time_since_epoch().count()));

    std::uniform_int_distribution<unsigned> jitter(0, 250);

    std::this_thread::sleep_for(
        std::chrono::milliseconds(delay + jitter(rng)));
}
```

Why jitter? Without it, thousands of gateways that lose connectivity together may retry on the same exponential schedule and create synchronized load spikes.

Threading model

```
Collector thread(s) --> bounded in-memory handoff --> WAL writer
                        |
                        v
                    durable records
                        |
Sender thread <--- Batcher/Reader <-----+-----+
```

Rule: RAM queues may improve throughput, but durability comes from the WAL, not from an in-memory queue.

11. Interview Walkthrough - 5 to 7 Minutes

Step 1 - clarify: "I want to establish device count, samples/sec, record size, maximum offline duration, latency target, loss tolerance and ordering requirement."

Step 2 - state the main constraint: "The intermittent WAN means edge durability is the first design problem. I will not make telemetry collection depend on cloud availability."

Step 3 - draw HLD: Sensor -> Collector -> Persistent WAL -> Batch/Compress -> WAN -> Cloud Ingestor -> Durable Event Stream -> Processing -> TSDB/Archive.

Step 4 - delivery semantics: "I prefer at-least-once delivery with idempotent cloud processing. Each record has deviceId + sequenceNo; an ACK advances a durable edge checkpoint."

Step 5 - scale: "Partition the event stream by deviceId to preserve per-device order while scaling horizontally."

Step 6 - attack the design: explain WAN down, crash, lost ACK, DB down, disk full and mass reconnect.

Step 7 - close with operations/security: mTLS/device certificates, schema versioning, queue depth, oldest unsent event, consumer lag and end-to-end latency.

One-line memory map

Intermittent WAN	-> WAL
Crash	-> recovery from WAL
Lost ACK	-> retry + dedup
High throughput	-> batch + compression
Cloud burst	-> durable event stream
Ordering	-> partition by device
Cloud slow	-> backpressure
Reconnect storm	-> backoff + jitter + throttling
Disk full	-> explicit priority/retention policy

This is the reasoning chain to communicate. Technology names come after the requirements and failure model.